



8 de octubre de 2026

COMPARACIÓN DE PUNTAJES EN MIPS

ALLAN BECERRA Y JEREMY LLERENA

UCOM250 – Organización y Arquitectura de Computadores
2026



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA
COMPARACIÓN DE PUNTAJES EN MIPS

Contenido

Descripción	2
Análisis	2
Implementación	3
Conclusiones	5
Bibliografía	5



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA
COMPARACIÓN DE PUNTAJES EN MIPS

Descripción

Al grupo 7 se le asignó el escenario **comparación de puntajes**. Dos jugadores, A y B, terminan una partida y sus puntajes quedan guardados en memoria: el jugador A obtuvo 85 puntos y el jugador B obtuvo 72. El problema consiste en calcular por cuántos puntos supera el jugador A al jugador B y determinar si los dos obtuvieron el mismo puntaje.

El programa debe cargar los puntajes desde la memoria, hacer los cálculos en los registros, guardar los resultados otra vez en la memoria y, al final, mostrarlos por pantalla con un mensaje que indique si hubo empate o no.

Análisis

Los datos del escenario se declaran en la sección .data del programa. La Tabla 1 muestra qué representa cada uno y el valor que tiene antes y después de la ejecución.

Tabla 1 Datos del programa

Dato	Valor inicial	Valor final	Propósito
dato1	85	85	Puntaje del jugador A
dato2	72	72	Puntaje del jugador B
resultado1	0	13	Diferencia de puntajes A – B
resultado2	0	0	1 si ambos tienen el mismo puntaje, 0 si no

Para resolver el escenario se necesitan las operaciones de la Tabla 2. Los resultados esperados son una diferencia de 13 puntos ($85 - 72$) y un 0 en la comparación, porque los puntajes no son iguales.

Tabla 2 Operaciones requeridas

Datos	Propósito	Operación requerida	Instrucción MIPS
dato1 → \$t0	Cargar el puntaje A	Carga	lw
dato2 → \$t1	Cargar el puntaje B	Carga	lw
Resultado 1 (\$t2)	Diferencia $85 - 72 = 13$	Resta	sub
Resultado 2 (\$t3)	¿Mismo puntaje? $85 \neq 72 \rightarrow 0$	Comparación	seq
Mensaje final	Elegir el mensaje según \$t3	Salto condicional	beq
Resultados	Guardar en resultado1 y resultado2	Almacenamiento	sw
Pantalla	Mostrar los resultados	Salida	syscall

UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO

FACULTAD DE INGENIERÍA

COMPARACIÓN DE PUNTAJES EN MIPS

Implementación

La solución se desarrolló en lenguaje ensamblador MIPS y se probó en el simulador mipsy web. Se parte de la versión base de la entrega anterior (version_base/programa_base.s), que ya cargaba los puntajes, calculaba la diferencia, hacía la comparación y guardaba los resultados en memoria. En la versión final (version_final/programa_final.s) se agregó la salida por pantalla y una decisión con beq. La Figura 1 muestra el código completo.



```

mipsy web
[Load] [Save] [Run] [Reset] [Kill] [Step Back] [Step Next] [Download]

source      decompiled      data
-----
Untitled (unsaved file changes)

1  # -----
2  # Comparacion de puntajes - Grupo 7 (version final)
3  # Integrantes: Allan Becerra y Jeremy Llerena
4  # Simulador: mipsy web
5  # -----
6
7  .data
8  dato1: .word 85 # puntaje jugador A
9  dato2: .word 72 # puntaje jugador B
10 resultado1: .word 0 # diferencia jugadorA - jugadorB
11 resultado2: .word 0 # 1 si ambos tienen el mismo puntaje, 0 si no
12
13 msg_dif: .asciiz "Diferencia de puntajes (A - B): "
14 msg_igual: .asciiz "Mismo puntaje (1 = si, 0 = no): "
15 msg_empate: .asciiz "Resultado: los jugadores empataron.\n"
16 msg_distin: .asciiz "Resultado: los jugadores tienen puntajes diferentes.\n"
17
18 .text
19 .globl main
20
21 main:
22 # ----- 1. Cargar datos desde memoria hacia registros -----
23 lw $t0, dato1 # $t0 = puntajeA (85)
24 lw $t1, dato2 # $t1 = puntajeB (72)
25
26 # ----- 2. Resta: diferencia jugadorA - jugadorB -----
27 sub $t2, $t0, $t1 # $t2 = 85 - 72 = 13
28
29 # ----- 3. Comparacion: mismo puntaje? -----
30 seq $t3, $t0, $t1 # $t3 = 1 si son iguales, 0 si no (85 != 72 -> 0)
31
32 # ----- 4. Guardar resultados en memoria -----
33 sw $t2, resultado1 # resultado1 = 13
34 sw $t3, resultado2 # resultado2 = 0
35
36 # ----- 5. Mostrar resultados por pantalla -----
37 li $v0, 4 # syscall 4: imprimir texto
38 la $a0, msg_dif
39 syscall
40 li $v0, 1 # syscall 1: imprimir entero
41 move $a0, $t2 # diferencia (13)
42 syscall
43 li $v0, 11 # syscall 11: imprimir caracter
44 li $a0, '\n' # salto de linea
45 syscall
46
47 li $v0, 4
48 la $a0, msg_igual
49 syscall
50 li $v0, 1
51 move $a0, $t3 # resultado de la comparacion (0)
52 syscall
53 li $v0, 11
54 li $a0, '\n'
55 syscall
56
57 # ----- 6. Control de flujo con beq -----
58 beq $t3, $zero, diferentes # si $t3 = 0, los puntajes no son iguales
59
60 li $v0, 4 # $t3 = 1: empate
61 la $a0, msg_empate
62 syscall
63 j fin
64
65 diferentes:
66 li $v0, 4 # $t3 = 0: puntajes diferentes
67 la $a0, msg_distin
68 syscall
69
70 fin:
71 # ----- 7. Terminar el programa -----
72 li $v0, 0
73 jr $ra
74

```

Figura 1 Código de la versión final en mipsy web

El programa está organizado en los siguientes bloques, marcados con comentarios en el código:

UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA
COMPARACIÓN DE PUNTAJES EN MIPS

Datos (.data): se declaran dato1 = 85, dato2 = 72, resultado1 y resultado2 en 0, y los textos que se muestran en pantalla.

Carga: con lw los puntajes pasan de la memoria a los registros \$t0 y \$t1.

Resta: sub \$t2, \$t0, \$t1 calcula la diferencia y deja 13 en \$t2.

Comparación: seq \$t3, \$t0, \$t1 deja 1 en \$t3 si los puntajes son iguales y 0 si no. En este caso queda 0.

Almacenamiento: con sw los valores de \$t2 y \$t3 se guardan en resultado1 y resultado2.

Salida por pantalla: con syscall 4 se imprime cada texto, con syscall 1 cada número y con syscall 11 el salto de línea.

Control de flujo: beq \$t3, \$zero, diferentes salta a la etiqueta diferentes cuando \$t3 vale 0. Si valiera 1, se mostraría el mensaje de empate.

Después de ejecutar el programa, el panel de registros muestra \$t0 = 85, \$t1 = 72, \$t2 = 13 y \$t3 = 0. En el segmento de datos, desde la dirección 0x10010000, los mismos valores aparecen en hexadecimal: 55 (85), 48 (72), 0D (13) y 00 (0), lo que confirma que los resultados quedaron guardados en memoria (Figura 2).

mipsy web

Load

Save

Run

Reset

Kill

Step Back

Step Next

Download

MIPS Docs

About

source		decompiled		data	
Untitled					
Data segment					
0x10010000	55 00 00 00	48 00 00 00	0d 00 00 00	00 00 00 00	U \0 \0 \0 H \0 \0 \0 \r \0 \0 \0 \0 \0 \0 \0
0x10010010	44 69 66 65	72 65 6e 63	69 61 20 64	65 20 70 75	D i f f e r e n c i a d e p u
0x10010020	6e 74 61 6a	65 73 20 28	41 20 2d 20	42 29 3a 20	n t a j e s (A - B) i
0x10010030	00 44 69 73	6d 6f 20 70	75 6e 74 61	6a 65 20 28	\0 H i s m o p u n t a j e (
0x10010040	31 20 3d 20	73 69 2c 20	30 20 3d 20	6e 6f 29 3a	1 = s i , \0 = n o) :
0x10010050	20 00 52 65	73 75 6c 74	61 64 6f 3a	20 6c 6f 73	\0 R e s u l t e d o : l o s
0x10010060	20 6a 75 67	61 64 6f 72	65 73 20 65	6d 70 61 74	j u g a d o r e s e w p a t
0x10010070	61 72 6f 6e	2e 00 00 52	65 73 75 6c	74 61 64 6f	a r o n - \n \0 R e s u l t e d o
0x10010080	3a 20 6c 6f	73 20 6a 75	67 61 64 6f	72 65 73 20	: l o s j u g a d o r e s
0x10010090	74 69 65 6e	65 6e 20 70	75 6e 74 61	6a 65 73 20	t i e n e n p u n t a j e s
0x100100a0	64 69 66 65	72 65 6e 74	65 73 2e 0a	00 -- --	d i f f e r e n t e s - \n \0 - -
0x100100b0	-- -- --	-- -- --	-- -- --	-- -- --	- - - - - - - - - - - -

used registers		all registers	
Read	Write	Register	Value
		\$zero	0
		\$at	0x1001000c
		\$v0	17
		\$a0	0
		\$a1	0
		\$t0	85
		\$t1	72
		\$t2	13
		\$t3	0
		\$s0	0x7fffffff
		\$fp	0x7fffffff
		\$ra	0x8000000c

Figura 2 Segmento de datos y registros después de la ejecución

Por último, la Figura 3 muestra la salida del programa. Se imprime la diferencia, el resultado de la comparación y el mensaje que eligió beg.

I/O	Mipsy Output - (0)
I/O Diferencia de puntajes (A - B): 13 Mismo puntaje (1 = si, 0 = no): 0 Resultado: los jugadores tienen puntajes diferentes.	

Figura 3 Resultado mostrado por pantalla



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA
COMPARACIÓN DE PUNTAJES EN MIPS

Conclusiones

Al principio nos costó entender que en MIPS los datos no se usan directamente desde la memoria. Primero hay que pasarlos a un registro con lw y al final devolverlos con sw. Cuando vimos los valores cambiar en el panel de registros y después en el segmento de datos, entendimos mucho mejor cómo viaja la información que cuando lo vimos solo en clase.

Lo que más trabajo nos dio fue la versión final, sobre todo mostrar los resultados en pantalla y usar beq para elegir el mensaje. Tuvimos que revisar la documentación del simulador para saber qué número de syscall usar en cada caso y ejecutar el programa paso a paso varias veces hasta que el salto funcionó como esperábamos.

También nos confundió que el segmento de datos mostrara los valores en hexadecimal. Pasarlos a decimal (55 es 85, 48 es 72 y 0D es 13) nos ayudó a comprobar que el programa guardaba bien los resultados.

Si tuviéramos que hacer la actividad otra vez, primero anotaríamos en papel qué registro guarda cada dato y probaríamos desde el inicio con dos puntajes iguales, para revisar los dos caminos del programa.

Bibliografía

Patterson, D. A., & Hennessy, J. L. (2021). *Computer organization and design: The hardware/software interface* (6.^a ed.). Morgan Kaufmann.

University of New South Wales. (s. f.). *MIPS instruction set*. Recuperado el 8 de octubre de 2026, de <https://cgi.cse.unsw.edu.au/~cs1521/current/resources/mips-guide.html>

University of New South Wales. (s. f.). *mipsy web* [Simulador MIPS]. Recuperado el 8 de octubre de 2026, de <https://cgi.cse.unsw.edu.au/~cs1521/mipsy/>